

Selling Price Calculation Methods — Impact Analysis

Date: 2026-06-18 **Status:** Design proposal — ready for review **Scope:** HMS pharmacy pricing system (high-level overview only; deep dives follow in the per-section punch lists)

TL;DR

The HMS should let each item pick one of two cost methods for selling-price calculation:

- **HIGHEST** (current) — $\text{Base Cost} = \text{MAX}(\text{Stock.pricePerUnit}) \text{ WHERE } \text{qty} > 0$
- **AVERAGE_WEIGHTED** — $\text{Base Cost} = \text{SUM}(\text{qty} \times \text{pricePerUnit}) / \text{SUM}(\text{qty})$

Profit margins, manual price overrides, and patient-group logic are unchanged; only the **base-cost selection** switches.

Single chokepoint. Every selling price flows through one repo method → one service → one REST route. All five billing modules (OPD / IPD / ED / HD / Cathlab) snapshot the price at write time and never recompute on read. The new method therefore affects **only new bills**. Historical view prices never move.

Backwards compatible. Postgres enum default **HIGHEST** preserves the current behaviour for every existing row. The rollout is silent for users who don't change an item's method.

Current flow (today)

```
StockRepository.getHighestPurchasedPrice(itemId)           - stock.repository.ts:478
└─ SELECT 1 FROM Stock
   WHERE itemId = $1 AND qty > 0
   ORDER BY pricePerUnit DESC
   LIMIT 1
└─ hardcoded — "highest" is the only option

PharmacyPricingService.getMaximumRetailPriceForItem()     - pharmacy-pricing.service.ts:13
└─ calls getHighestPurchasedPrice
└─ applies manualPrice override (LOCAL/FOREIGNER/INSURANCE/GOVERNMENT) if set
└─ otherwise: round(highestCost + highestCost × sellingPriceGroup.{patient}ProfitPercentage / 100)

GET /api/maximum-retail-price?itemId=...&patientGroup=... - api/(pharmacy)/maximum-retail-price/route.ts:7
└─ calls the service above
└─ returns { success, result, error }
```

2 sites duplicate this logic (already a smell):

- Client-side mirror: `pharmacy/pharmacy-sale/features/utils.ts:82` (`getMaximumRetailPrice({highestPurchasedPrice, ...})`)
- Change-selling-price default fallback: `shared/pharmacy/repositories/change-selling-price-repository.ts:39-86` (re-runs the highest-cost query inline when no manual price exists)

Files that BREAK on switch (must change)

#	File:line	Why
1	<code>app/(dashboard)/shared/pharmacy/repositories/stock.repository.ts:478</code>	Hardcoded <code>ORDER BY pricePerUnit DESC LIMIT 1</code>
2	<code>app/(dashboard)/shared/pharmacy/services/pharmacy-pricing.service.ts:13</code>	Calls the broken repo above
3	<code>app/(dashboard)/shared/pharmacy/repositories/change-selling-price-repository.ts:39-86</code>	Re-implements the same hardcoded highest-cost query

#	File:line	Why
4	<code>app/api/(pharmacy)/maximum-retail-price/route.ts:7-18</code>	Zod schema + service call — needs to pass cost method through
5	<code>app/api/(common)/inventory/stock-highest-purchased-price/route.ts:1-27</code>	Hardcoded in URL name + response; either keep HIGHEST-only or add <code>?method=</code> param
6	<code>app/api/(pharmacy)/latest-price-list/route.ts:1-32</code> → <code>latest-price-repository.ts:57-125</code>	Raw SQL uses <code>MAX(purchased_price_per_unit) ... WHERE qty > 0</code> — needs HIGHEST vs <code>SUM(qty*p)/SUM(qty)</code> branch
7	<code>app/(dashboard)/pharmacy/pharmacy-sale/features/utils.ts:82-142</code>	Hardcoded <code>highestPurchasedPrice</code> param name (semantic rename)
8	<code>app/(dashboard)/pharmacy/stock/features/api/get-highest-purchased-price.ts:5-26</code>	React Query key <code>["highestPurchasedPrice", itemId]</code> — rename to <code>["baseCost", itemId, costMethod]</code> for cache hygiene
9	<code>app/(dashboard)/pharmacy/change-selling-price/features/components/change-selling-price-columns.tsx:54</code>	UI hardcodes <code>const hintLabel = "Latest"</code> — switch to 'Avg Cost' for AVERAGE_WEIGHTED items
10	<code>app/(dashboard)/pharmacy/pharmacy-sale/features/hooks/usePharmacySaleCalculations.ts</code>	Cache key needs cost method (transitively — see #8)
11	<code>prisma/schema.prisma</code> — Item model	Needs new enum column <code>costMethod</code> <code>CostMethod @default(HIGHEST) @map("cost_method")</code>
12	<code>common/items/features/components/item-form.tsx:65, 453-476</code> + <code>schemas/item-form-schema.ts:91</code>	Add the selector UI next to <code>sellingPriceGroupId</code>
13	<code>common/items/[id]/page.tsx:82</code>	Detail page should render the chosen method
14	Zod schemas in <code>common-selling-price-schema.ts:7-39</code> and <code>latest-price-list-schema.ts:7-19</code>	Pass <code>costMethod</code> through to history rows and latest-price list rows

Files AFFECTED but not broken (write-time snapshots stay valid)

All five billing modules call `/api/maximum-retail-price` only when **creating** a new bill line. They snapshot `maximumRetailPrice` + `amount` into the persisted row and **never recompute on read**. The new method only changes the price of **new bills**.

Module	Caller	Verdict
OPD	<code>opd/opd-billing/features/components/opd-billing-pharmacy.tsx:71-72</code> reads snapshot	New bills only
IPD ward-service	<code>ipd/features/components/ward-service/ward-service-pharmacy-sale-item-row.tsx:73-78</code> calls MRP API	New bills only
IPD billing	<code>ipd-billing/features/components/ipd-billing-pharmacy-sale.tsx:73</code> (wraps ward row)	New bills only
ED	<code>ed/features/components/ed-billing-pharmacy-sales.tsx:365-391</code> calls MRP API	New bills only

Module	Caller	Verdict
HD	<code>hd/features/hooks/use-bind-form.tsx:185</code> reads snapshot — no MRP fetch	Inherited from upstream sale
Cathlab	<code>cathlab/features/components/cathlab-pharmacy-sale-item-row.tsx:88-115</code> calls MRP API	New bills only

Read paths that consume the snapshot (no change needed, just be aware):

- `ipd-billing/features/components/daily-bill-*.tsx` (8 files: list, ct-section, ward-services, cathlab, proxy-bill, receipt x2 layouts, returns)
- `pharmacy/pharmacy-sale-ipd/`, `pharmacy/sale-return/`, `pharmacy/sale-return-ipd/`
- `pharmacy/pharmacy-transfer/features/components/*transfer-detail.tsx`

No module is "would change historical view prices". All reads are snapshot-based.

Sale-return / transfer quirks

- `sale-return/` and `sale-return-ipd/` **don't recompute** — they hydrate from `pharmacy_sale_items.maximum_retail_price`. New returns work the same; old returns show what was sold.
- `pharmacy-transfer/` reads `saleItem?.maximumRetailPrice` for display. **Unaffected.**

Schema decision (recommended)

```
enum CostMethod {
  HIGHEST
  AVERAGE_WEIGHTED
}

model Item {
  // ...existing fields...
  costMethod CostMethod @default(HIGHEST) @map("cost_method")
  // ...
}

// Optional but recommended – freeze audit data:
model SellingPriceChangeHistory {
  // ...existing fields...
  costMethod CostMethod? @map("cost_method")
  baseCostSnapshot Decimal? @map("base_cost_snapshot") @db.Decimal(12, 4)
}
```

Justification:

- **Per-item, not per-group.** `SellingPriceGroup` is shared by many items — wrong granularity.
- Cost is derived from `Stock` rows for one item; `costMethod` must follow the item.
- The cost query at `stock.repository.ts:478` already filters by `itemId`, so no extra join.
- Storing `costMethod` on the history row lets a retroactive audit answer "which method was active when this price was recorded?" — otherwise changing an item's method later silently rewrites the audit story.
- Storing `baseCostSnapshot` on the history row freezes the actual base cost used (defends against later cost-method or stock-mix changes).
- Optional: add `costMethod` to `ItemManualPrice` too, to remember which method the user locked in when they set the override.

Next migration: `20260618xxxxxx_add_cost_method_to_items`. No backfill needed (default `'HIGHEST'` preserves current behaviour for every existing row).

Most recent pricing-related migrations (context):

- `20260405224915_add_manual_price_override_tables` — created `item_manual_prices` and `selling_price_change_histories`
- `20250304043612_add_item_type_column_in_items_table` — last `Item` column change

- 20250203041344_add_company_batch_no_to_stocks_table — last Stock column change

UI placement

- Per-item selector on app/(dashboard)/common/items/[id]/page.tsx — add a costMethod row next to the existing sellingPriceGroup row at page.tsx:82.
- Form: extend common/items/features/components/item-form.tsx (the existing sellingPriceGroupId field is at lines 453-476) with a Mantine Select.
- Schema entry: common/items/features/schemas/item-form-schema.ts:91 (next to sellingPriceGroupId).

RBAC: no new permission needed. This is a base-cost derivation concern, not a selling-price-list concern. It rides on the existing Edit:Item / Edit:Item Management permission that already gates the item form.

Tests (zero exist today for pricing internals)

Test file	Verdict
pharmacy/latest-price-list/__tests__/latest-price-repository.node.test.ts	needs mock shape update + new HIGHEST vs AVERAGE_WEIGHTED branch test
pharmacy/latest-price-list/__tests__/features/components/price-list-table.dom.test.tsx	verify any new column renders
pharmacy/latest-price-list/__tests__/features/api/get-latest-price-list-api.node.test.ts	assert new fields if exposed
All other existing tests (sale-return, ipd-daily-bill-price-updates, opd-billing-pharmacy-sale, ot-emr-services-payload, etc.)	pass unchanged — none touch cost-method code paths

New tests to write:

- StockRepository: pair of HIGHEST + AVERAGE_WEIGHTED pricing tests (mock stock.findFirst / stock.findMany + stock.aggregate).
- PharmacyPricingService: parameterized by cost method x manual price present/absent x each patient group.
- ChangeSellingPriceRepository: default fallback uses the correct method.
- change-selling-price-columns.tsx: label switches on cost method.
- common-selling-price-schema / latest-price-list-schema: Zod validation tests.

Docs (new section needed)

- hms-docs/api/manifest.yaml — add a note about the new item field.
- hms-docs/api/paths/common-items.yaml — document the new costMethod enum value.
- hms-docs/api/paths/trpc-items.yaml — same.
- hms-docs/api/schemas/common-items.yaml — add the costMethod enum on the Item schema.
- hms-docs/api/paths/common-selling-price-groups.yaml — note that costMethod is per-item, not per-group.
- New file: hms-docs/api/paths/stock-base-cost.yaml documenting the raw cost semantics (HIGHEST vs AVERAGE_WEIGHTED) — could also live inline in common-items.yaml.
- No existing doc discusses cost method today.

React Query cache invalidation

Two keys must include costMethod so a per-item method switch and a new GRN both trigger refetch:

Old key	New key
["highestPurchasedPrice", itemId]	["baseCost", itemId, costMethod]
["maximumRetailPrice", itemId, patientGroup]	["maximumRetailPrice", itemId, patientGroup, costMethod]

`getHighestPurchasedPrice` staleTime is currently 5 min with `refetchOnMount: false` — verify it stays short enough that a freshly-posted GRN + method flip is visible.

Server vs client split (and how to clean it up)

Server (authoritative): `stock.repository.ts:478` → `pharmacy-pricing.service.ts:13` → `api/(pharmacy)/maximum-retail-price/route.ts:26`. The server reads `Item.costMethod` directly, so the client doesn't need to send it — the server is the source of truth.

Client (mirror, mostly unused for the calculation itself): `get-maximum-retail-price.api.ts` + the `getMaximumRetailPrice` helper in `pharmacy-sale/features/utils.ts:82`. The `usePharmacySaleCalculations` hook's effect then writes the server-computed value back into the form.

Switching methods only needs a server change. But the client helper should be renamed from `highestPurchasedPrice` to `baseCost` to remove the ambiguity at the call site.

Localization strings to watch

- `change-selling-price/features/components/change-selling-price-columns.tsx:54` — `const hintLabel = "Latest";` → must become `item.costMethod === 'AVERAGE_WEIGHTED' ? 'Avg Cost' : 'Latest'`.
- `pharmacy/latest-price-list/page.tsx:8` — `PageWrapper page="Lastest Selling Price"` (sic, "Lastest") and `<Title order={4}>Latest Price List</Title>` at `price-list-table.tsx:73`. "Latest" here means "most recent manual/auto price", not "highest" — probably safe but flag for product.

No other UI string is coupled to "highest".

Risk summary

- **Lowest risk:** the price snapshot pattern means **no historical view prices change**. Existing bills, sale returns, transfers, IPD daily bills, and receipts are all read-from-snapshot and stay byte-identical.
- **Lowest code churn:** the change is concentrated in 1 repo method + 1 service + 1 REST route + 1 raw-SQL repo + 1 schema column + 1 UI selector. The rest of the system is "new bills use the new method".
- **Medium risk:** the raw SQL in `latest-price-list` repo needs a parameterized branch. The change is mechanical but the query is the production list-page query and must be tested with realistic data.
- **Lowest test impact:** zero existing tests touch the pricing internals — nothing to rewrite, but new tests must be written for the new branch.
- **Localization risk:** one hardcoded "Latest" string in `change-selling-price-columns.tsx:54` needs to become `'Avg Cost'` for `AVERAGE_WEIGHTED` items. No other UI string is coupled to "highest".

Suggested phasing

1. **Phase 1 — schema + repo** (½ day)
 - Add `CostMethod` enum + `costMethod` column on `Item` (default `HIGHEST`).
 - Add `getWeightedAveragePurchasedPrice()` repo method + unit tests.
 - **Zero behaviour change** — every existing row defaults to `HIGHEST`.
2. **Phase 2 — service + REST route** (½ day)
 - Thread `costMethod` through `getMaximumRetailPriceForItem` and `/api/maximum-retail-price`.
 - Server reads `Item.costMethod` directly; client doesn't send it.
3. **Phase 3 — UI selector** (½ day)
 - Add the dropdown to the item form (`item-form.tsx:453–476` area) + detail page.
 - RBAC ride-along; no new permission.
4. **Phase 4 — report raw SQL** (½ day)
 - Extend `latest-price-list` repo to branch on `costMethod` in both items CTE and COUNT query.
 - Update test mocks.
5. **Phase 5 — change-selling-price default** (¼ day)
 - Update `change-selling-price-repository.ts:39–86` to use the right method.
 - Update "Latest/Avg Cost" label in `change-selling-price-columns.tsx:54`.
6. **Phase 6 — docs** (¼ day)

- Update `hms-docs/api/...` paths and schemas; add `paths/stock-base-cost.yaml` if needed.

Total estimate: ~2.5 days, mostly test-writing and SQL review.

Open questions for product / architecture review

1. **Default for existing items.** Postgres default `HIGHEST` preserves current behaviour. Confirm — or do we want to flip new items to `AVERAGE_WEIGHTED` by default?
2. **Per-tenant vs per-hospital.** Today the schema is single-tenant on-prem. If multi-tenant arrives, does the `costMethod` choice become per-tenant or stay per-item? (My recommendation: stay per-item; the calculation is local to one item's stock.)
3. **History audit depth.** Do we want `costMethod` + `baseCostSnapshot` on `SellingPriceChangeHistory` for a complete audit trail? (My recommendation: yes — both are cheap and protect against retroactive interpretation issues.)
4. **Stock-highest-purchased-price endpoint fate.** Keep it as a `HIGHEST`-only legacy endpoint? Deprecate? Rename to `stock-base-price?method=?` (My recommendation: deprecate in favour of `/api/maximum-retail-price` reading `Item.costMethod`; the only known caller is the change-selling-price column hint.)
5. **Out-of-stock handling.** When `qty = 0` for all batches of an item, the current query returns 0. For `AVERAGE_WEIGHTED` this is the same (sum over empty set). Confirm: zero stock → price 0 → can't sell, which is the existing UX.